UNIVERSITÀ di VERONA
Dipartimento di INFORMATICA

Discrete Optimization and Decision Making

Kernel Search

Master's degrees in Data Science - Artificial Intelligence - Mathematics
Academic Year 2024/2025

Romeo Rizzi (Dept. of Computer Science, Univ. of Verona)
Roberto Zanotti (Dept. of Clinical and Experimental Sciences, Univ. of Brescia)

Mixed Integer Linear Programming Problem (MIP)

$$\max \ \mathbf{c}^{\mathsf{T}}\mathbf{x}$$

$$\mathbf{A}\mathbf{x} \leq \mathbf{b}$$

$$x_j \geq 0 \quad j \in \mathcal{C}$$

$$x_j \in \{0, 1\} \quad j \in \mathcal{B}$$

$$x_j \geq 0 \quad \text{integer } j \in \mathcal{I}$$

- When $\mathcal{I} = \emptyset$ we have 0–1 MIP

Mixed Integer Linear Programming Problem (MIP)

$$\max \ \mathbf{c}^{\mathsf{T}}\mathbf{x}$$

$$\mathbf{A}\mathbf{x} \leq \mathbf{b}$$

$$x_j \geq 0 \quad j \in \mathcal{C}$$

$$x_j \in \{0, 1\} \quad j \in \mathcal{B}$$

$$x_j \geq 0 \quad \text{integer } j \in \mathcal{I}$$

- When $\mathcal{I} = \emptyset$ we have 0–1 MIP
- Binary variables model decisions of selecting/refusing items

What is our goal?

The focus is put on developing a

HEURISTIC FRAMEWORK

for obtaining high-quality solutions to

GENERAL MIP problems

in a reasonable amount of time

Combinatorial Optimization: Solution Approaches

Traditionally, four groups of methods:

1) Exact methods
2) Constructive heuristics (greedy algorithms)
3) Local search methods (steepest descent/ascent algorithms)
4) Metaheuristic methods

Combinatorial Optimization: Solution Approaches

Traditionally, four groups of methods:

1) Exact methods
2) Constructive heuristics (greedy algorithms)
3) Local search methods (steepest descent/ascent algorithms)
4) Metaheuristic methods

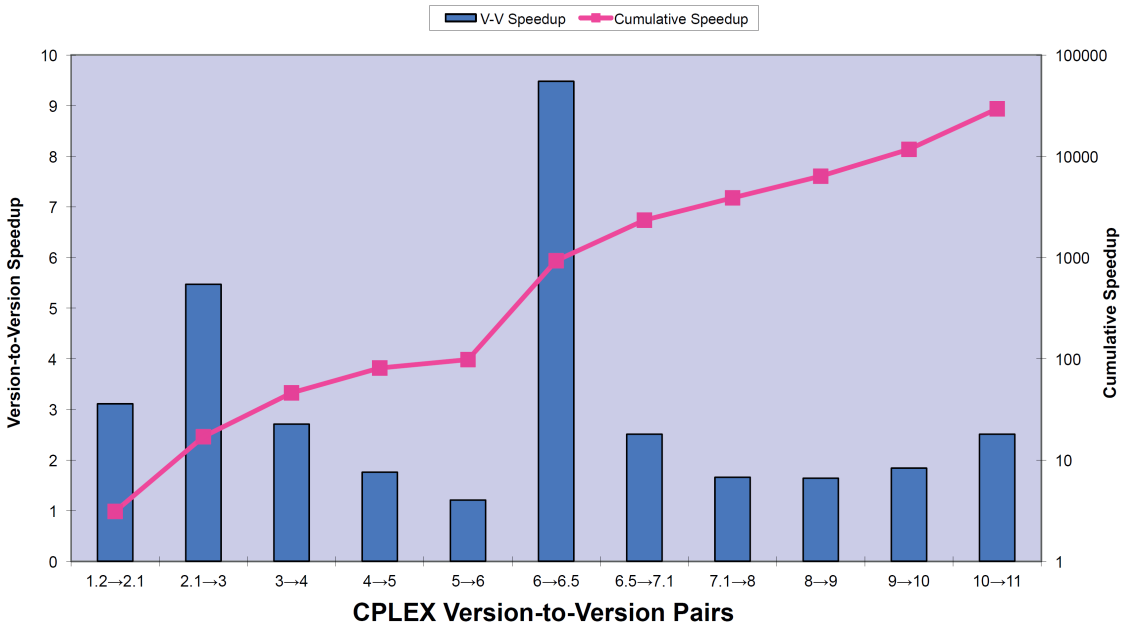
More recently, a new group has appeared:

5) Matheuristics

... intended as hybridization of heuristic and exact solution techniques

Exact methods for MIP problems

- General
- More and more powerful (Branch-and-Cut, Branch-and-Price)
- Off-the-shelf commercial solvers (Gurobi, CPLEX)



Inadequate for instances of medium/large size

Metaheuristics

Local Search

1. Choose an initial solution s^0 (constructive heuristic); set $k := 0$;
2. **While** (stopping criterion not satisfied) **repeat**
   a. Explore $N(s^k)$ (the set of neighbor solutions of s^k);
   b. Choose a solution $s^{k+1} \in N(s^k)$
   c. Set $k := k + 1$

Usually:

- Step 1: initial solution $\rightarrow$ generation fast and easy
- Step 2.a: exploration methods $\rightarrow$ First improvement, Best improvement
- Stopping criterion: an ascent (descent) method stops as soon as there is no better solution than s^k in $N(s^k)$

Local Search gets stuck in local optima

$\Downarrow$

need for new methods able to exploit the interplay between
DIVERSIFICATION/INTENSIFICATION

Metaheuristics

Local Search gets stuck in local optima

$\Downarrow$

need for new methods able to exploit the interplay between
DIVERSIFICATION/INTENSIFICATION

A meta-heuristic is a high-level problem-independent algorithmic framework that provides a set of guidelines or strategies to develop heuristic optimization algorithms.

(Sörensen and Glover, 2013)

Metaheuristics

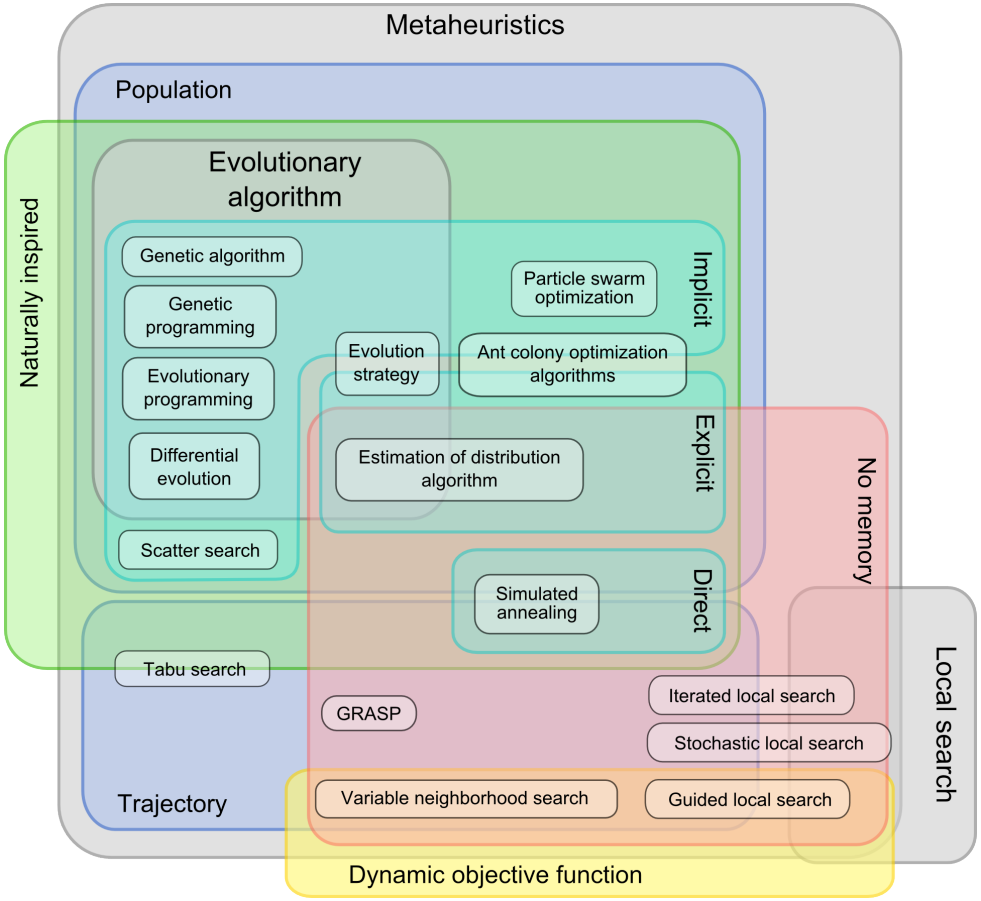


Figura 1: Euler Diagram of metaheuristics

Metaheuristics: not everything works

Thousands of new metaheuristics are being created each year, usually nature-inspired:

Metaheuristics: not everything works

Thousands of new metaheuristics are being created each year, usually nature-inspired:

- Donkey Optimization (10.1016/j.jcde.2019.04.004)

Metaheuristics: not everything works

Thousands of new metaheuristics are being created each year, usually nature-inspired:

- Donkey Optimization (10.1016/j.jcde.2019.04.004)
- FIFA World Cup Optimization (10.1007/s40313-016-0242-6)

Metaheuristics: not everything works

Thousands of new metaheuristics are being created each year, usually nature-inspired:

- Donkey Optimization (10.1016/j.jcde.2019.04.004)
- FIFA World Cup Optimization (10.1007/s40313-016-0242-6)
- Musician Optimization (10.1177/003754970107600201)

Metaheuristics: not everything works

Thousands of new metaheuristics are being created each year, usually nature-inspired:

- Donkey Optimization (10.1016/j.jcde.2019.04.004)
- FIFA World Cup Optimization (10.1007/s40313-016-0242-6)
- Musician Optimization (10.1177/003754970107600201)
- Reincarnation Optimization (10.1109/cinti.2010.5672231)

Metaheuristics: not everything works

Thousands of new metaheuristics are being created each year, usually nature-inspired:

- Donkey Optimization (10.1016/j.jcde.2019.04.004)
- FIFA World Cup Optimization (10.1007/s40313-016-0242-6)
- Musician Optimization (10.1177/003754970107600201)
- Reincarnation Optimization (10.1109/cinti.2010.5672231)
- Roach Infestation Optimization (10.1109/sis.2008.4668317)

A lot more are out there...

- The EC Bestiary (github.com/fcampelo/EC-Bestiary)

The use of meta-heuristics

- Needed where exact methods are inadequate
- Often designed for a specific problem
- May require great implementation effort

And if the problem slightly changes?

Frequently, the heuristic needs to be re-designed...

Heuristics versus exact methods

Can we combine

the strength of exact methods

with the strength of heuristics?

How?

- What kind of MIPs?
- Which heuristic scheme?
- Tailored or general?
- How?

Matheuristics

Matheuristics

Matheuristics combine exact mathematical programming algorithms with metaheuristics in a cooperative way

- Doerner, K. F., and Schmid, V. (2010). Survey: matheuristics for rich vehicle routing problems. In International Workshop on Hybrid Metaheuristics (pp. 206-221). Springer, Berlin, Heidelberg.
- Ball, M. O. (2011). Heuristics based on mathematical programming. Surveys in Operations Research and Management Science, 16(1), 21-38.

… often still tailored algorithms

Strategic general purpose methods

General matheuristics:

- based on MIP formulation
- valid for any MIP problem (or at least for classes of MIP problems)
- exploiting a MIP solver (frequently as a black-box)

Major approaches:

1. Local Branching [2003]
2. RINS [2005]
3. Kernel Search [2010]
4. Proximity Search [2014]

Local Branching

Given a feasible solution $\bar{x}$, define a k-opt neighborhood of $\bar{x}$ as:

$$\Delta(x, \bar{x}) := \sum_{j \in \mathcal{B}:\ \bar{x}_j = 1} (1 - x_j) + \sum_{j \in \mathcal{B}:\ \bar{x}_j = 0} x_j \leq k$$

The method:

- is embedded into a Branch-and-Bound: explores solution subspaces through a branching framework
- favors early updating of the incumbent solution
- is designed to improve heuristic behaviour of the MIP solver

Relaxation Induced Neighborhood Search (RINS)

At each node of a branch-and-bound tree:

- $\bar{x}$ = the incumbent integer solution
- x^{LP} = the optimal LP solution

The RINS algorithm (MIN problem):

1) fix all the variables that have the same value in $\bar{x}$ and x^{LP}
2) add a cut-off constraint $c^T x \leq c^T \bar{x} - \theta$
3) solve a restricted MIP problem on the remaining variables

Parameters:

- How often to run the method (every k nodes)
- Node limit for the restricted MIP problem

Proximity Search

Repeat until time limit is reached (MIN PROBLEM):

1) Add the cut-off constraint $c^T x \leq c^T \bar{x} - \theta$, where $\bar{x}$ is a feasible solution and $\theta > 0$ a cut-off tolerance

Proximity Search

Repeat until time limit is reached (MIN PROBLEM):

1) Add the cut-off constraint $c^T x \leq c^T \bar{x} - \theta$, where $\bar{x}$ is a feasible solution and $\theta > 0$ a cut-off tolerance
2) Replace the objective function with a proximity one:

$$\Delta(x, \bar{x}) := \sum_{j \in \mathcal{B}:\ \bar{x}_j = 1} (1 - x_j) + \sum_{j \in \mathcal{B}:\ \bar{x}_j = 0} x_j$$

Proximity Search

Repeat until time limit is reached (MIN PROBLEM):

1) Add the cut-off constraint $c^T x \leq c^T \bar{x} - \theta$, where $\bar{x}$ is a feasible solution and $\theta > 0$ a cut-off tolerance
2) Replace the objective function with a proximity one:

$$\Delta(x, \bar{x}) := \sum_{j \in \mathcal{B}:\ \bar{x}_j = 1} (1 - x_j) + \sum_{j \in \mathcal{B}:\ \bar{x}_j = 0} x_j$$

3) Solve the new problem (not necessarily to optimality) by using a MIP solver and obtain a new incumbent x^* (possible void solution)

Proximity Search

Repeat until time limit is reached (MIN PROBLEM):

1) Add the cut-off constraint $c^T x \leq c^T \bar{x} - \theta$, where $\bar{x}$ is a feasible solution and $\theta > 0$ a cut-off tolerance
2) Replace the objective function with a proximity one:

$$\Delta(x, \bar{x}) := \sum_{j \in \mathcal{B}:\ \bar{x}_j = 1} (1 - x_j) + \sum_{j \in \mathcal{B}:\ \bar{x}_j = 0} x_j$$

3) Solve the new problem (not necessarily to optimality) by using a MIP solver and obtain a new incumbent x^* (possible void solution)
4) Recenter $\Delta(x, \cdot)$ by setting $\bar{x} := x^*$, and/or update θ

Proximity Search vs Local Branching 1/2

Proximity Search		Local Branching
$\min \Delta(x, \tilde{x})$	Objective Function	$\min f(x)$
$f(x) \leq f(\tilde{x}) - \theta$	Cutoff constraint / Neighborhood constraint	$\Delta(x, \tilde{x}) \leq k$
$g(x) \leq 0$	Other constraints	$g(x) \leq 0$

Proximity Search vs Local Branching 2/2

Proximity Search

$$\min \Delta(x, \tilde{x})$$

$$f(x) \leq f(\tilde{x}) - \theta$$

$$g(x) \leq 0$$

Local Branching

$$\min f(x)$$

$$\Delta(x, \tilde{x}) \leq k$$

$$g(x) \leq 0$$

Neighborhood constraint

Cutoff constraint

Other constraints

Kernel Search

Kernel Search: a few preliminary considerations

When solving MIP problems we can observe that:

- often in an optimal solution there are a few non-zero variables
- often basic variables in the LP-relaxation are good predictors of non-zero variables in an optimal MIP solution
- often reduced costs are good predictors of the likelihood of a non-basic variable to be zero/non-zero in a MIP optimal solution

Kernel Search: Main Idea

Identify a KERNEL SET

$\Downarrow$

a set of promising variables
(likely to be non-zero in an optimal solution)

Kernel Search: Main Idea

Identify a KERNEL SET

$\Downarrow$

a set of promising variables
(likely to be non-zero in an optimal solution)

Construct a sequence of RESTRICTED MIP problems

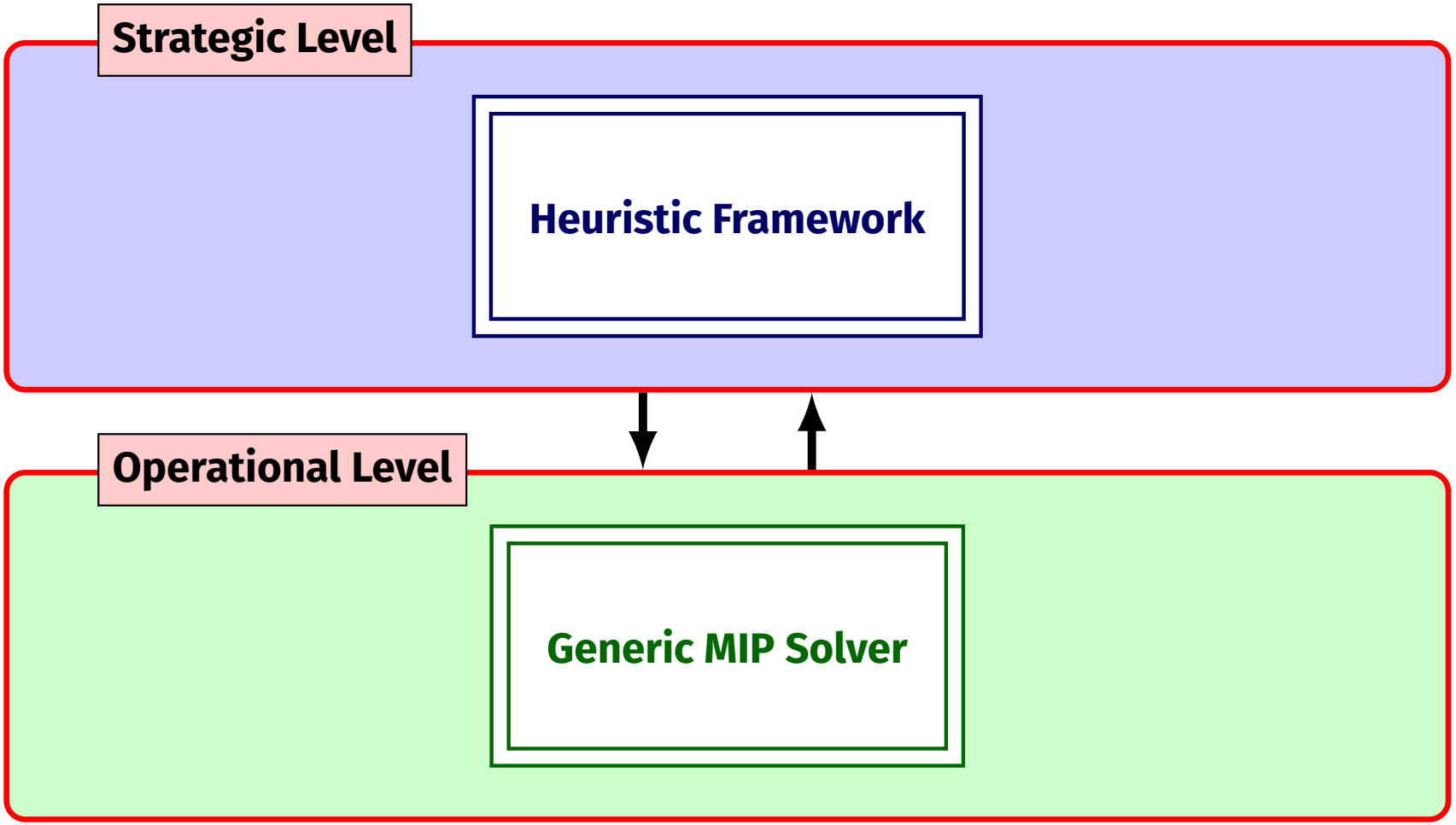
$\Downarrow$

including Kernel Set and some more variables

RESTRICTED:

- not too large (avoid computational effort)
- not too small (allow "correlated" variables to be jointly selected)

Kernel Search: Main Structure



The framework

Main features:

- Simple to implement
- Tough work in charge of the MIP solver
- Trade-off quality/efficiency controlled by a few parameters

Kernel Search: the Heuristic Framework

PHASE 1
(Initialization)

INITIAL KERNEL SET
Select promising variables in the Kernel Set

INITIAL SOLUTION
Solve a restricted MIP on the Kernel Set

Kernel Search: the Heuristic Framework

PHASE 1
(Initialization)

INITIAL KERNEL SET
Select promising variables in the Kernel Set

INITIAL SOLUTION
Solve a restricted MIP on the Kernel Set

PHASE 2
(Improvement)

SOLVE RESTRICTED PROBLEMS
Solve a sequence of restricted MIP problems

KERNEL SET UPDATE
Iteratively extend the Kernel Set

Kernel Search: PHASE 1 (INITIALIZATION)

1) Solve the LP relaxation of the initial problem
2) Sort variables (Sorting Criterion)
3) Construct Initial Kernel Set Λ
4) Partition remaining variables into a sequence $\{B_i | \ i = 1, ..., nb\}$ of buckets (Buckets Construction)
5) Solve restricted MIP(Λ) problem:
   optimal solution $\mathbf{x}^*$ with value w^*

PHASE 1: LP solution

1) Solve LP relaxation

$$\max \ \mathbf{c}^{\mathsf{T}}\mathbf{x} \tag{1}$$

$$\mathbf{A}\mathbf{x} = \mathbf{b} \tag{2}$$

$$\mathbf{x} \geq 0 \tag{3}$$

Optimal solution: $\mathbf{x} = (\mathbf{x}_B, \mathbf{x}_N)$

PHASE 1: LP solution

1) Solve LP relaxation

$$\max \ \mathbf{c}^{\mathsf{T}}\mathbf{x} \tag{1}$$

$$\mathbf{A}\mathbf{x} = \mathbf{b} \tag{2}$$

$$\mathbf{x} \geq 0 \tag{3}$$

Optimal solution: $\mathbf{x} = (\mathbf{x}_B, \mathbf{x}_N)$

2) Get value of the variables in the basis $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$

PHASE 1: LP solution

1) Solve LP relaxation

$$\max \ \mathbf{c}^{\mathbf{T}}\mathbf{x} \tag{1}$$

$$\mathbf{A}\mathbf{x} = \mathbf{b} \tag{2}$$

$$\mathbf{x} \geq 0 \tag{3}$$

   Optimal solution: $\mathbf{x} = (\mathbf{x}_B, \mathbf{x}_N)$

2) Get value of the variables in the basis $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$

3) Get value of reduced costs (variables out of the basis): $\mathbf{c}_N^T - \mathbf{c}_\mathbf{B}^T\mathbf{B}^{-1}\mathbf{N}$

PHASE 1: LP solution

1) Solve LP relaxation

$$\max \ \mathbf{c}^{\mathbf{T}}\mathbf{x} \tag{1}$$

$$\mathbf{A}\mathbf{x} = \mathbf{b} \tag{2}$$

$$\mathbf{x} \geq 0 \tag{3}$$

   Optimal solution: $\mathbf{x} = (\mathbf{x}_B, \mathbf{x}_N)$

2) Get value of the variables in the basis $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$

3) Get value of reduced costs (variables out of the basis): $\mathbf{c}_N^T - \mathbf{c}_\mathbf{B}^T\mathbf{B}^{-1}\mathbf{N}$

PHASE 1: LP solution

1) Solve LP relaxation

$$\max \ \mathbf{c}^{\mathbf{T}}\mathbf{x} \tag{1}$$

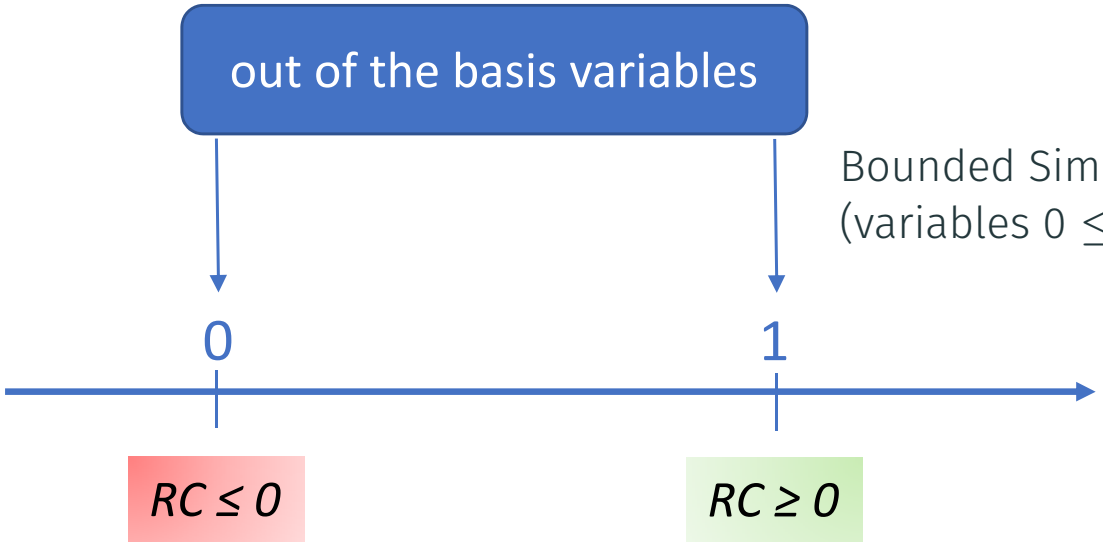
$$\mathbf{A}\mathbf{x} = \mathbf{b} \tag{2}$$

$$\mathbf{x} \geq 0 \tag{3}$$

Optimal solution: $\mathbf{x} = (\mathbf{x}_B, \mathbf{x}_N)$

2) Get value of the variables in the basis $\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$

3) Get value of reduced costs (variables out of the basis): $\mathbf{c}_N^T - \mathbf{c}_\mathbf{B}^T\mathbf{B}^{-1}\mathbf{N}$

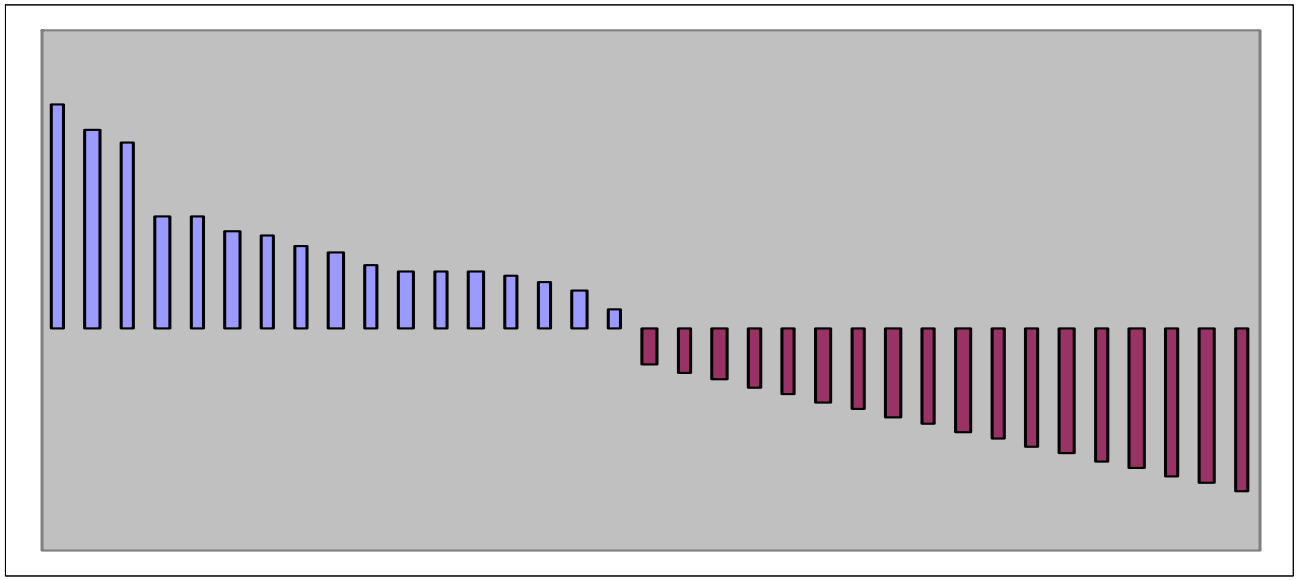


Bounded Simplex algorithm
(variables $0 \leq \mathbf{x} \leq 1$)

PHASE 1: Sorting Criterion

Sort variables according to:

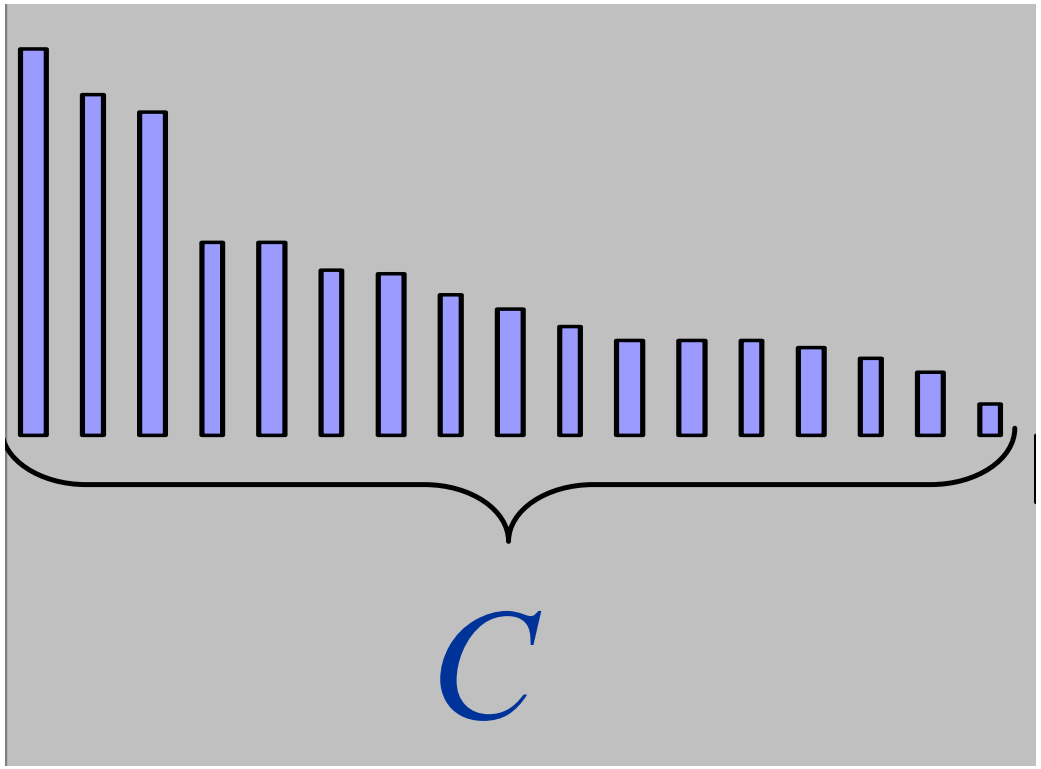
- NON INCREASING value of variables in LP relaxation solution
- NON DECREASING value of absolute reduced costs



$$\mathbf{c}^T\mathbf{x} = \mathbf{c}_B^T(\mathbf{B}^{-1}\mathbf{b} - \mathbf{B}^{-1}\mathbf{N}\mathbf{x}_N) + \mathbf{c}_N^T\mathbf{x}_N = \mathbf{c}_B^T\mathbf{B}^{-1}\mathbf{b} + (\mathbf{c}_N^T - \mathbf{c}_B^T\mathbf{B}^{-1}\mathbf{N})\mathbf{x}_N$$

PHASE 1: Kernel Set construction

Kernel Set Λ: select the first C variables



C = # variables with positive value in LP

PHASE 1: Initial solution

Solve the restricted problem MIP(Λ)

(the original MIP problem restricted to the Kernel Set Λ)

with CPLEX/GUROBI

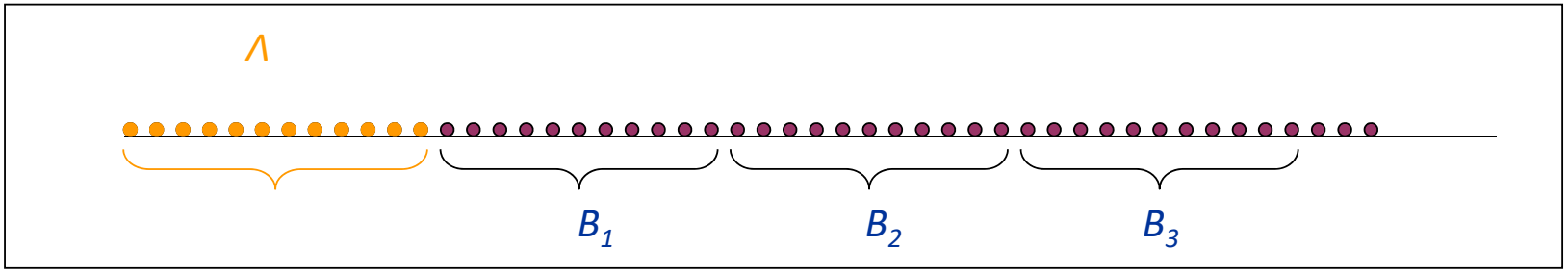
$$\Downarrow$$

optimal solution $\mathbf{x}^*$ and optimal value w^*

(if no solution found set $w^* := -\infty$)

PHASE 1: Buckets construction

Partition remaining variables into buckets



HOW?

Numerical example

The Multidimensional Multiple-Choice Knapsack Problem:

- Given
  - A collection $\mathcal{G} = \{G_1, G_2, \ldots, G_n\}$ of n disjoint sets of items
  - Each group G_i has n_i items (i, j), $j = 1, \ldots, n_i$
  - A set $M = \{1, 2, \ldots, m\}$ of resources
  - An availability $c_k > 0$ for each resource $k \in M$
  - A profit $p_{ij} > 0$ for each item $j \in G_i$ and $G_i \in \mathcal{G}$
  - A weight $w_{ij}^k \geq 0$ for each $k \in M$, $j \in G_i$ and $G_i \in \mathcal{G}$

Numerical example

The Multidimensional Multiple-Choice Knapsack Problem:

- Given
  - A collection $\mathcal{G} = \{G_1, G_2, \ldots, G_n\}$ of n disjoint sets of items
  - Each group G_i has n_i items (i, j), $j = 1, \ldots, n_i$
  - A set $M = \{1, 2, \ldots, m\}$ of resources
  - An availability $c_k > 0$ for each resource $k \in M$
  - A profit $p_{ij} > 0$ for each item $j \in G_i$ and $G_i \in \mathcal{G}$
  - A weight $w_{ij}^k \geq 0$ for each $k \in M$, $j \in G_i$ and $G_i \in \mathcal{G}$
- Choose one item for each group, maximizing the overall profit, without exceeding the resources

MMKP: the mathematical formulation

$$\text{Maximize} \sum_{i=1}^{n} \sum_{j=1}^{n_i} p_{ij} x_{ij} \tag{4}$$

subject to:

$$\sum_{i=1}^{n} \sum_{j=1}^{n_i} w_{ij}^{k} x_{ij} \leq c_k \qquad k = 1, \ldots, m, \tag{5}$$

$$\sum_{j=1}^{n_i} x_{ij} = 1 \qquad i = 1, \ldots, n, \tag{6}$$

$$x_{ij} \in \{0, 1\} \qquad (i, j) \in N, \tag{7}$$

Sorting by Reduced Costs: MMKP example

Instance MMKP: 5 groups, 5 items per group, 5 resources

c_1	c_2	c_3	c_4	c_5
14	20	20	23	18

Group	Item	p_{ij}	w_{ij}^1	w_{ij}^2	w_{ij}^3	w_{ij}^4	w_{ij}^5
1	1	129	3	4	4	8	1
1	2	182	1	5	3	7	7
1	3	183	9	4	2	0	4
1	4	219	4	0	2	7	9
1	5	229	8	5	1	4	4
2	1	113	0	2	3	7	0
2	2	166	2	3	2	3	8
2	3	214	0	5	8	9	7
2	4	217	3	6	5	6	6
2	5	219	5	2	3	6	5
...	...	...	...	...	...	...	...

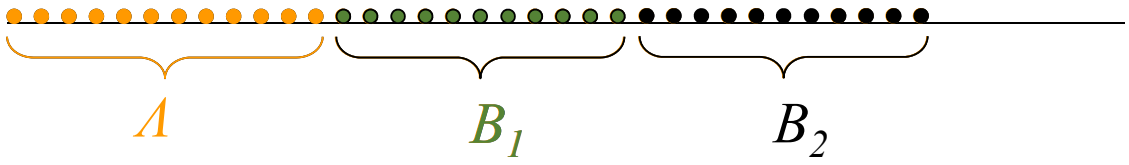
Group	Item	x^*_{LP}	RC	x^*_{ILP}
1	1	0	-20.09	0
1	2	0	-10.13	0
1	3	0	-47.13	0
1	4	0.76	0	1
1	5	0.24	0	0
2	1	0.05	0	0
2	2	0	-50.17	0
2	3	0.76	0	0
2	4	0	-14.40	0
2	5	0.18	0	1
3	1	0	-70.46	0
3	2	0.94	0	1
3	3	0.06	0	0
3	4	0	-16.28	0
3	5	0	-15.60	0
4	1	0	-35.46	0
4	2	1	3.19	1
4	3	0	-39.79	0
4	4	0	-28.00	0
4	5	0	0	0
5	1	0	-1.50	1
5	2	0	-12.46	0
5	3	0.41	0	0
5	4	0	-38.52	0
5	5	0.59	0	0

Group	Item	x^*_{LP}	RC	x^*_{ILP}
1	1	0	-20.09	0
1	2	0	-10.13	0
1	3	0	-47.13	0
1	4	0.76	0	1
1	5	0.24	0	0
2	1	0.05	0	0
2	2	0	-50.17	0
2	3	0.76	0	0
2	4	0	-14.40	0
2	5	0.18	0	1
3	1	0	-70.46	0
3	2	0.94	0	1
3	3	0.06	0	0
3	4	0	-16.28	0
3	5	0	-15.60	0
4	1	0	-35.46	0
4	2	1	3.19	1
4	3	0	-39.79	0
4	4	0	-28.00	0
4	5	0	0	0
5	1	0	-1.50	1
5	2	0	-12.46	0
5	3	0.41	0	0
5	4	0	-38.52	0
5	5	0.59	0	0

Group	Item	x^*_{LP}	RC	x^*_{ILP}
4	2	1	3.19	1
3	2	0.94	0	1
1	4	0.76	0	1
2	3	0.76	0	0
5	5	0.59	0	0
5	3	0.41	0	0
1	5	0.24	0	0
2	5	0.18	0	1
3	3	0.06	0	0
2	1	0.05	0	0
4	5	0	0	0
5	1	0	-1.50	1
1	2	0	-10.13	0
5	2	0	-12.46	0
2	4	0	-14.40	0
3	5	0	-15.60	0
3	4	0	-16.28	0
1	1	0	-20.09	0
4	4	0	-28.00	0
4	1	0	-35.46	0
5	4	0	-38.52	0
4	3	0	-39.79	0
1	3	0	-47.13	0
2	2	0	-50.17	0
3	1	0	-70.46	0

PHASE 2 (Improvement)

1) Set $i := 1$
2) Construct set $\Lambda \cup B_i$ (B_i current bucket)



3) Solve restricted problem MIP($\Lambda \cup B_i$) + some constraints
4) If MIP($\Lambda \cup B_i$) is feasible:
   - update incumbent integer solution ($\mathbf{x}^*$, w^*)
   - construct $\bar{\Lambda}_i := \{$selected variables in $B_i\}$
   - update Kernel Set: $\Lambda := \Lambda \cup \bar{\Lambda}_i$
5) If $i < nb$
   - set $i := i + 1$ and **go to** Step 2

otherwise STOP

PHASE 2: Solving the restricted MIP problems

To improve efficiency when solving each MIP($\Lambda \cup B_i$):

First constraint

value of the obj. function $\geq w^*$

Second constraint

$\sum_{j \in B_i} x_j \geq \mathbf{1}$

Solution of restricted MIP problems

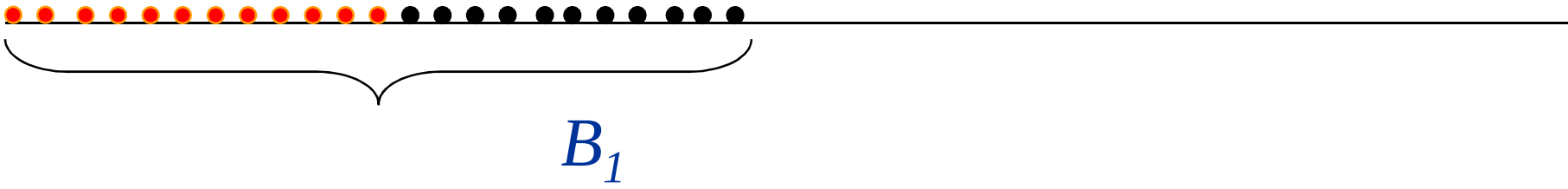
Solving each restricted problem MIP($\Lambda \cup B$)

has (possibly) two main objectives:

$$\Downarrow$$

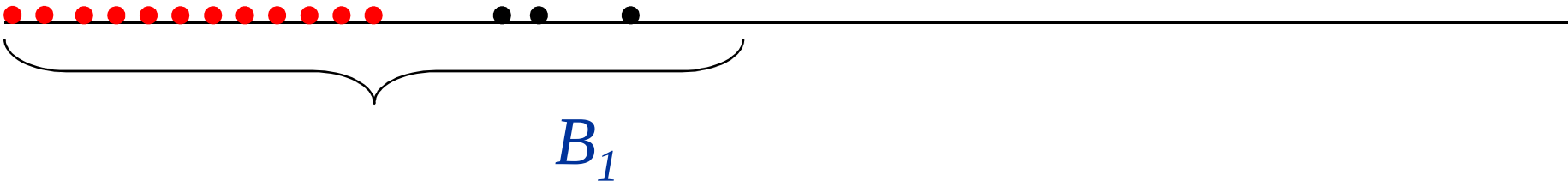
1. find an improving feasible solution
2. identify new variables to enter the Kernel Set

PHASE 2: Scrolling the buckets



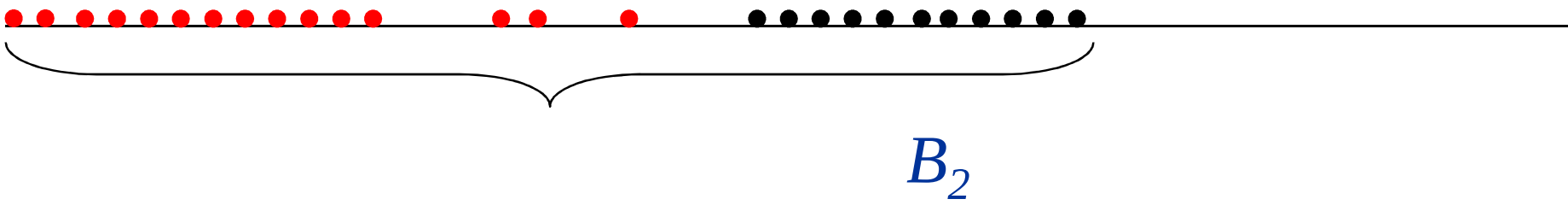
Restricted MILP

PHASE 2: Scrolling the buckets



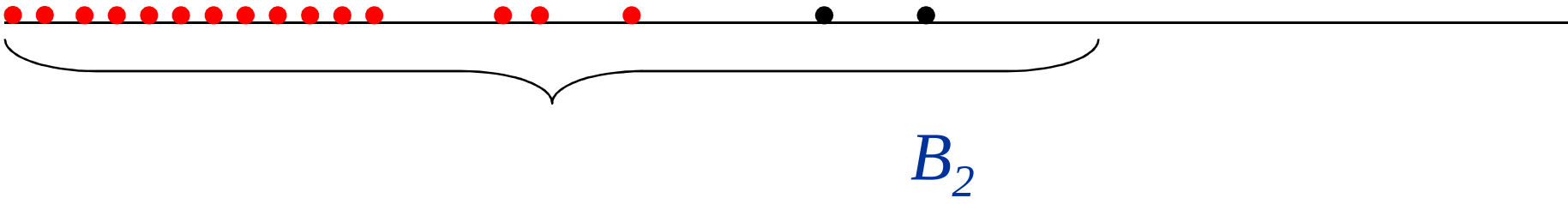
Updated kernel

PHASE 2: Scrolling the buckets



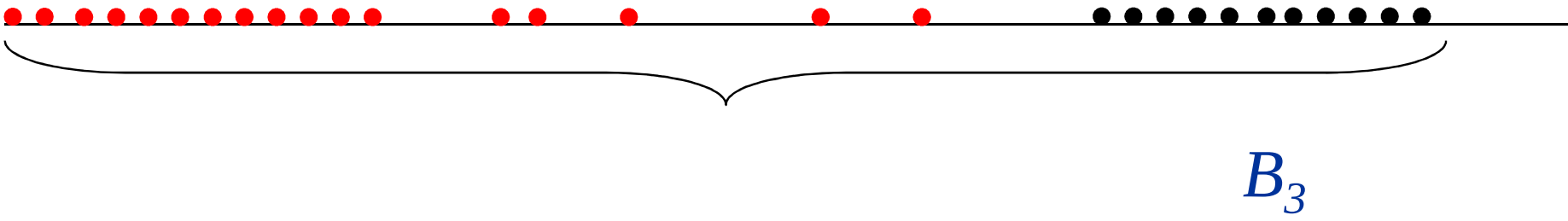
Restricted MILP

PHASE 2: Scrolling the buckets



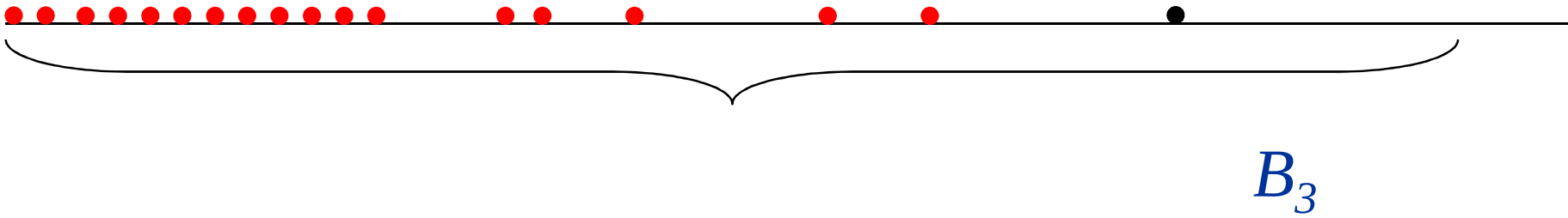
Updated kernel

PHASE 2: Scrolling the buckets



Restricted MILP

PHASE 2: Scrolling the buckets



Updated kernel

Critical aspects

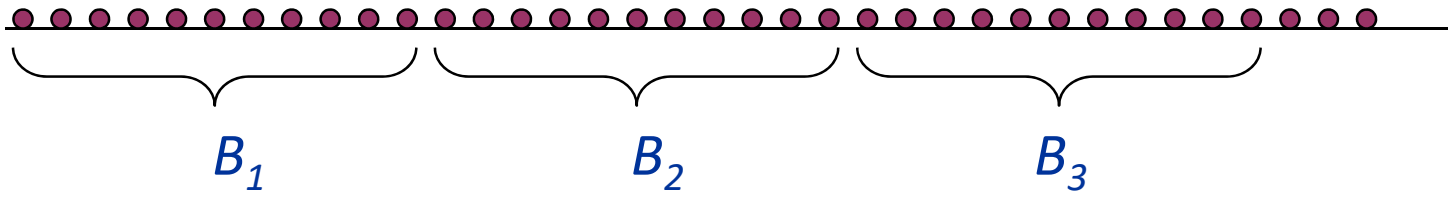
The Kernel Set increases monotonically (no variable is discharged) $\Rightarrow$ NEW VARIANTS

Critical aspects

The Kernel Set increases monotonically (no variable is discharged) $\Rightarrow$ NEW VARIANTS

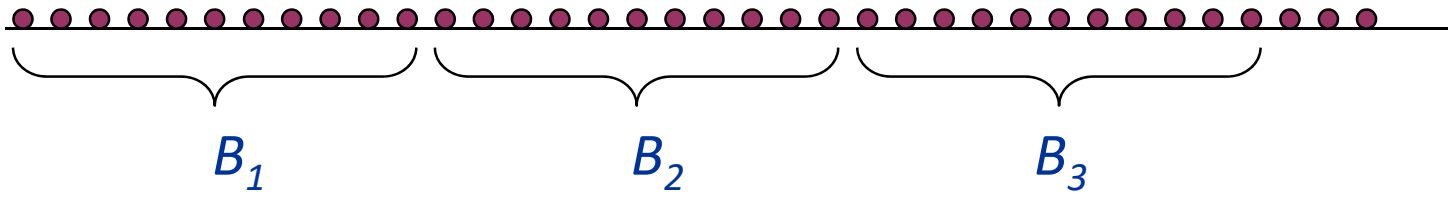
How do we deal with buckets construction?

Buckets construction



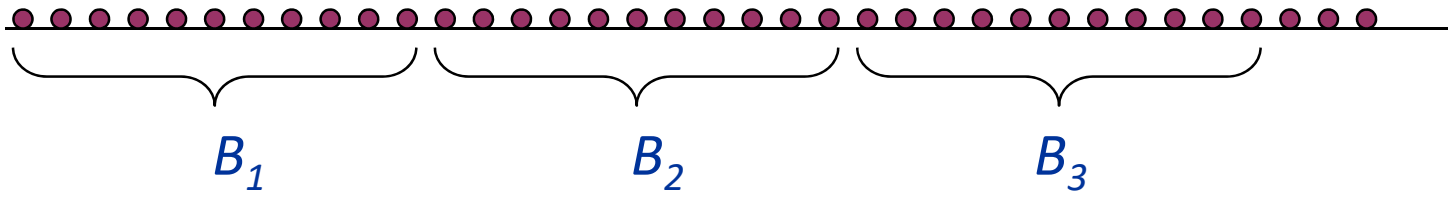
- What is the cardinality of each bucket? SMALL/LARGE

Buckets construction



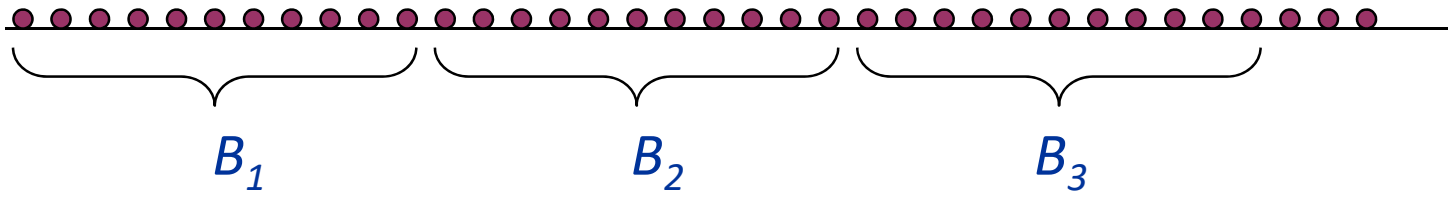
- What is the cardinality of each bucket? SMALL/LARGE
- How many buckets? A FEW/MANY

Buckets construction



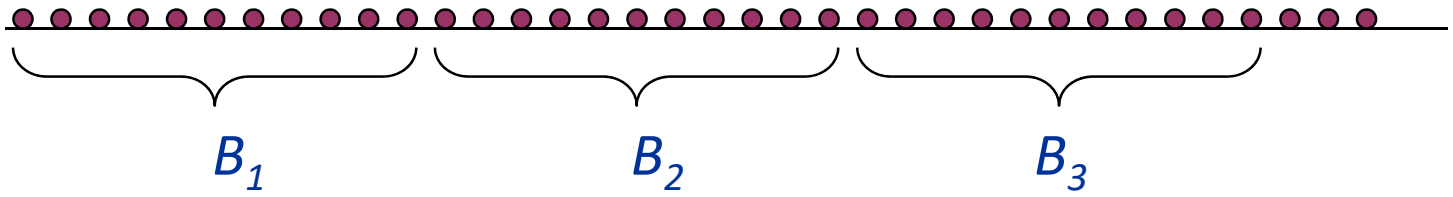
- What is the cardinality of each bucket? SMALL/LARGE
- How many buckets? A FEW/MANY
- Should bucket size vary? FIXED/VARIABLE

Buckets construction



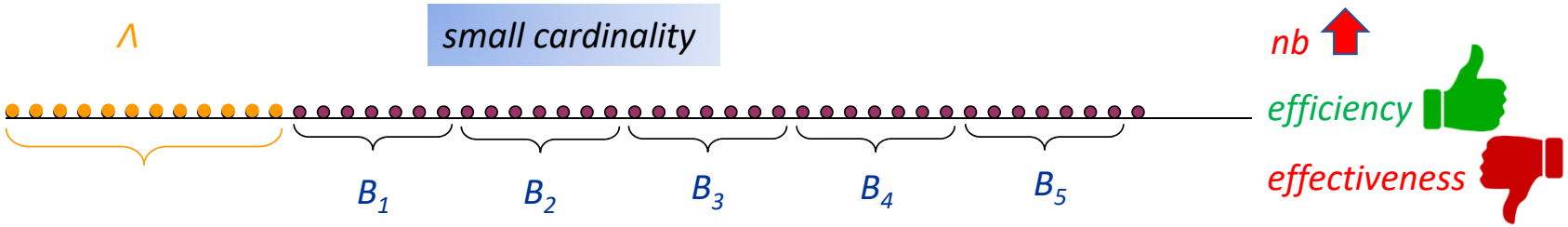
- What is the cardinality of each bucket? SMALL/LARGE
- How many buckets? A FEW/MANY
- Should bucket size vary? FIXED/VARIABLE
- Should buckets share common variables? DISJOINT/OVERLAPPING

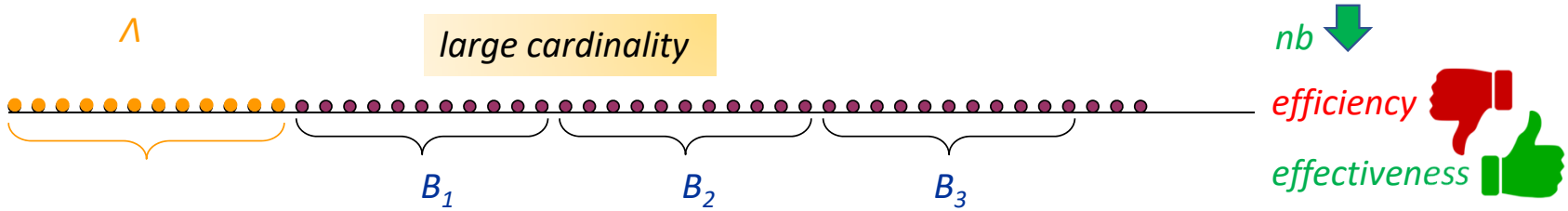
Buckets construction



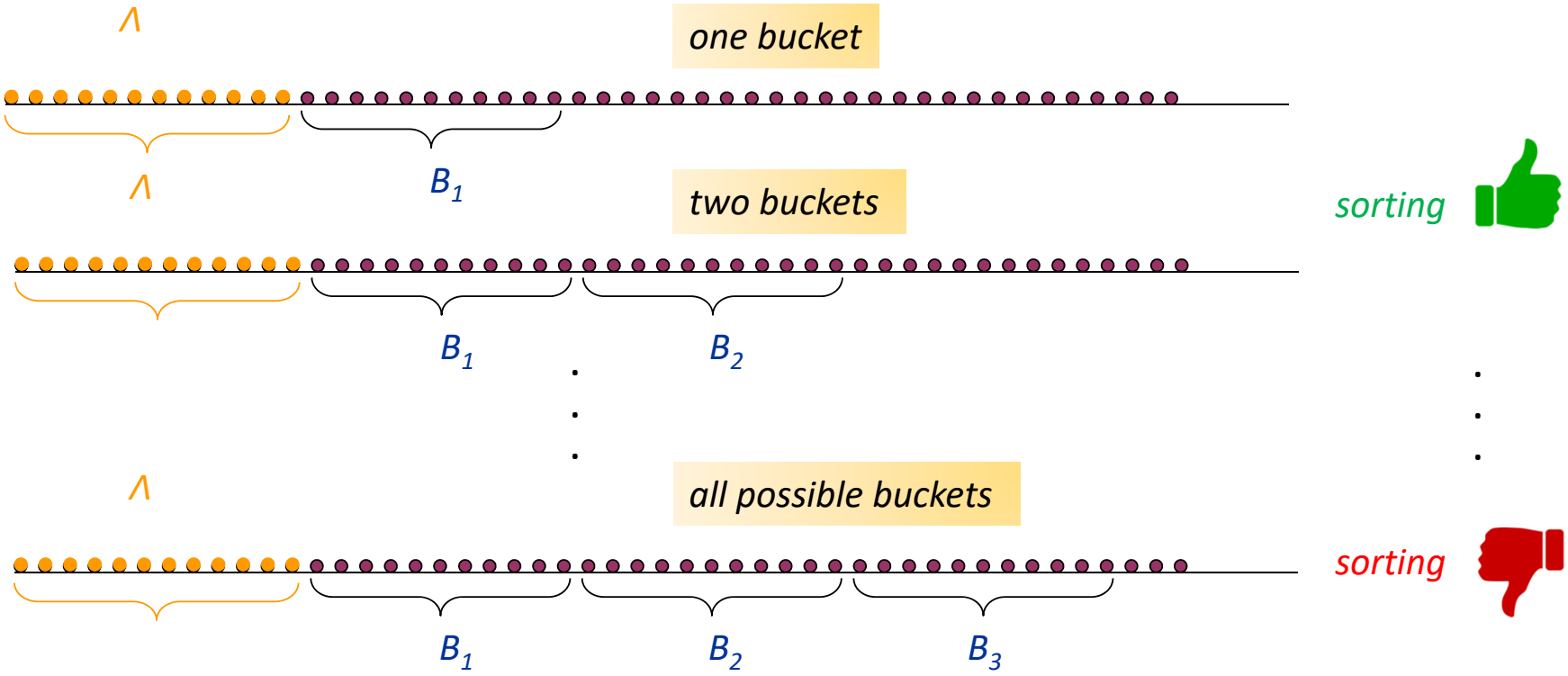
- What is the cardinality of each bucket? SMALL/LARGE
- How many buckets? A FEW/MANY
- Should bucket size vary? FIXED/VARIABLE
- Should buckets share common variables? DISJOINT/OVERLAPPING
- Should buckets be scrolled more than once? NO/YES

Bucket cardinality: small or large?



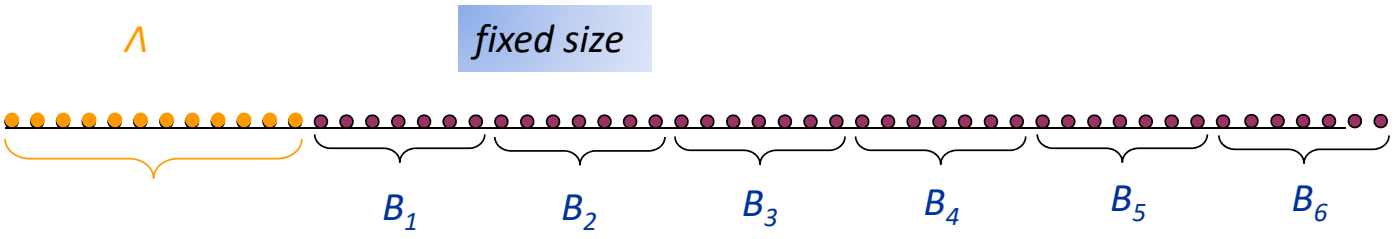


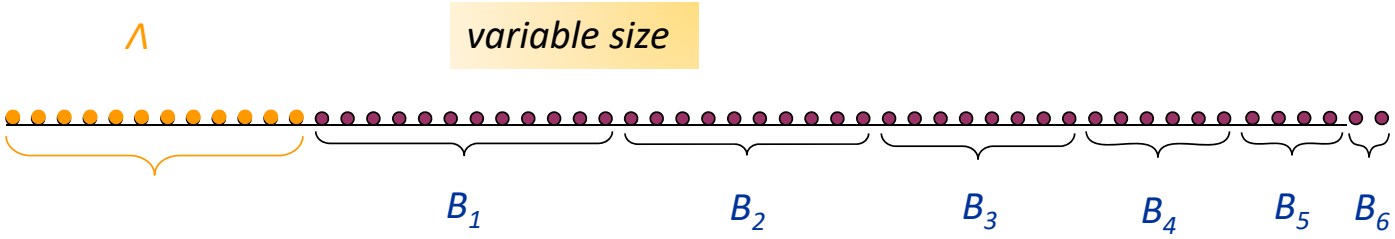
Number of buckets (nb): a few or many buckets?



Bucket size: fixed or variable?

Fixed bucket size: kC with $k > 0$



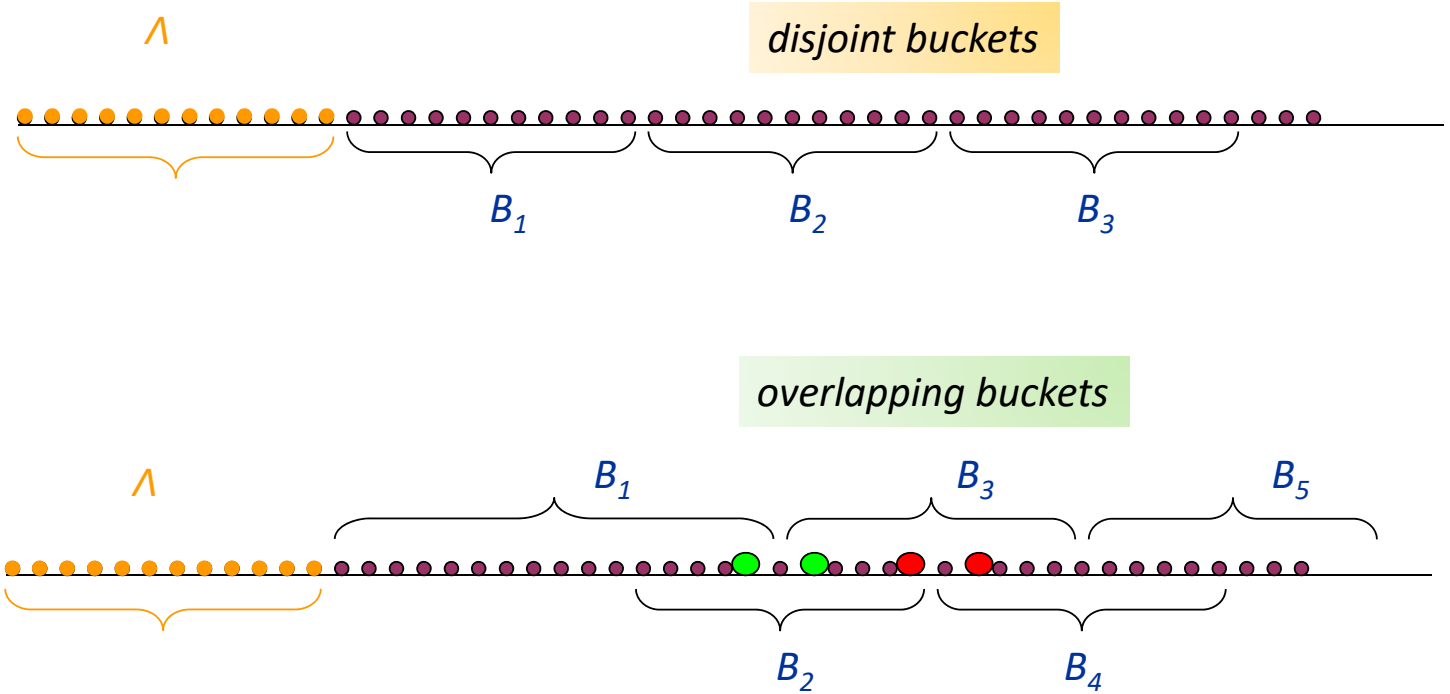


Variable bucket size: linearly decreasing, linearly increasing, others...

Example:$|B_1| = C$ $|B_i| = \max\{s_1, C - s_2(i-1)\}$ $i = 2, \ldots, nb$

Should buckets be disjoint sets or overlapping ones?

Disjoint buckets: $B_i \cap B_j = \emptyset \; \forall i \neq j$



Shifted buckets example: $|B_1| = 1.5C \; |B_i| = C \; i = 2, \ldots, nb$

What about

if we want to scroll buckets

more than once?

IKS

Iterative Kernel Search

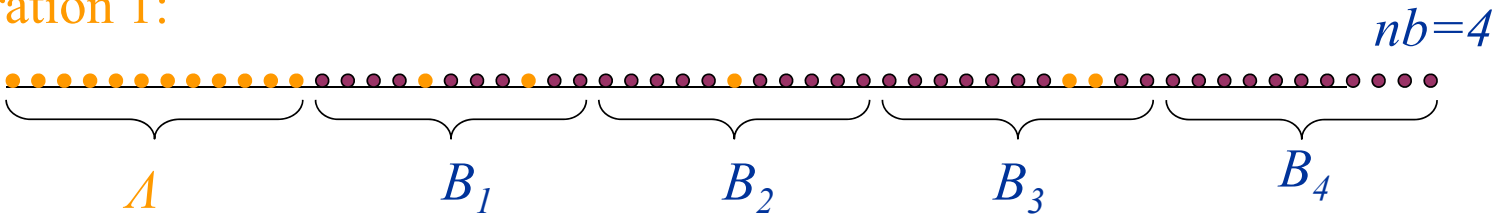
Buckets are scrolled more than once

Repeat until no more variables are added to Kernel Set:

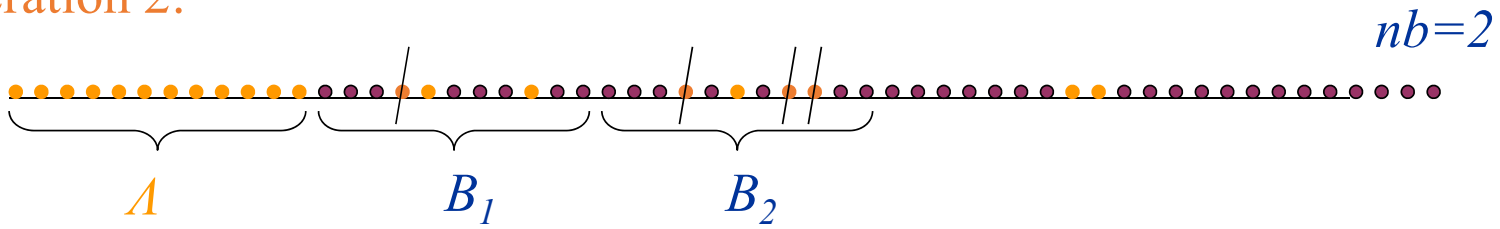
1) Call PHASE 1 + PHASE 2
2) Set $nb := q - 1$ where $q := \max\{i | \bar{\Lambda}_i \neq \emptyset\}$
3) Call PHASE 2
4) **If** a new variable entered the Kernel Set:
   - set nb to its initial value
   - Call PHASE 2 and **goto** Step 2

Iterative Kernel Search: an example

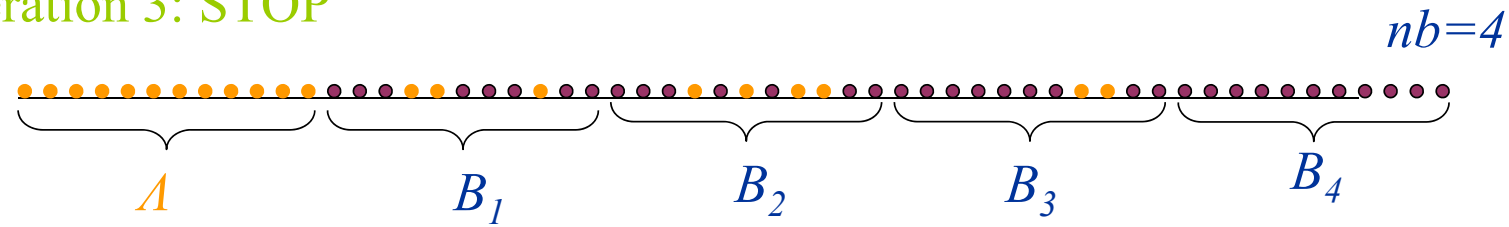
Iteration 1:



Iteration 2:



Iteration 3: STOP



Kernel Search (KS) vs other general purpose methods

- Local Branching vs KS

	LB	KS
problem	BIP	MIP
exact algorithm	yes	no
primal heuristic	yes	yes
stand-alone method	no	yes
required initial solution	yes	no

- RINS vs KS

	RINS	KS
problem	MIP	MIP
exact algorithm	no	no
primal heuristic	yes	yes
stand-alone method	no	yes
required initial solution	yes	no

- Proximity Search vs KS

	PS	KS
problem	BIP	MIP
exact algorithm	no	no
primal heuristic	yes	yes
stand-alone method	yes	yes
required initial solution	yes	no

Kernel Search: any drawbacks?

1) What if reduced costs do not provide a good signal?

Kernel Search: any drawbacks?

1) What if reduced costs do not provide a good signal?

2) What if Kernel Set grows too large?

Kernel Search: any drawbacks?

1) What if reduced costs do not provide a good signal?

2) What if Kernel Set grows too large?

3) Are there any good ways to identify correlation between variables?

Kernel Search: any drawbacks?

1) What if reduced costs do not provide a good signal?

2) What if Kernel Set grows too large?

3) Are there any good ways to identify correlation between variables?

4) Should we dynamically control the solution time of the restricted MILP problems?

New Variants

- Guastaroba G., Savelsbergh M. and Speranza M.G. (2017). Adaptive kernel search: A heuristic for solving mixed integer linear programs. European Journal of Operational Research, 263(3), 789-804.

  (Root node LP relaxation, Kernel Set iteratively increased to get feasibility; time required to solve the first restricted problem used to classify the instance).

- Hanafi, S., Mansini, R., Zanotti, R. (2020). The multi-visit team orienteering problem with precedence constraints. European Journal of Operational Research, 282(2), 515-529.

  (Selection/interchange of different sorting rules, dynamic procedure to control buckets and Kernel Set size, affinity concept among variables; use of Kernel Search as primal method)

- Lamanna, L., Mansini R., and Zanotti R. (2021). A Two-phase Kernel Search variant for the Multidimensional Multiple-choice Knapsack Problem. *European Journal of Operational Research, accepted*

  (Two Phases; Phase 1: feasibility-oriented; Phase 2: quality-oriented; internal tuning of parameters: SLOW/FAST instances)

Conclusions

Summarizing on Kernel Search

- LP relaxation solution used to determine Kernel Set and buckets
- Construction and solution of a sequence of small/medium size restricted MIP problems
- Successive bounds improvement
- Kernel Set monotonically increased (no item is discharged)